ĐỒ THỊ (GRAPH)

DATA STRUCTURES AND ALGORITHMS

L/O/G/O

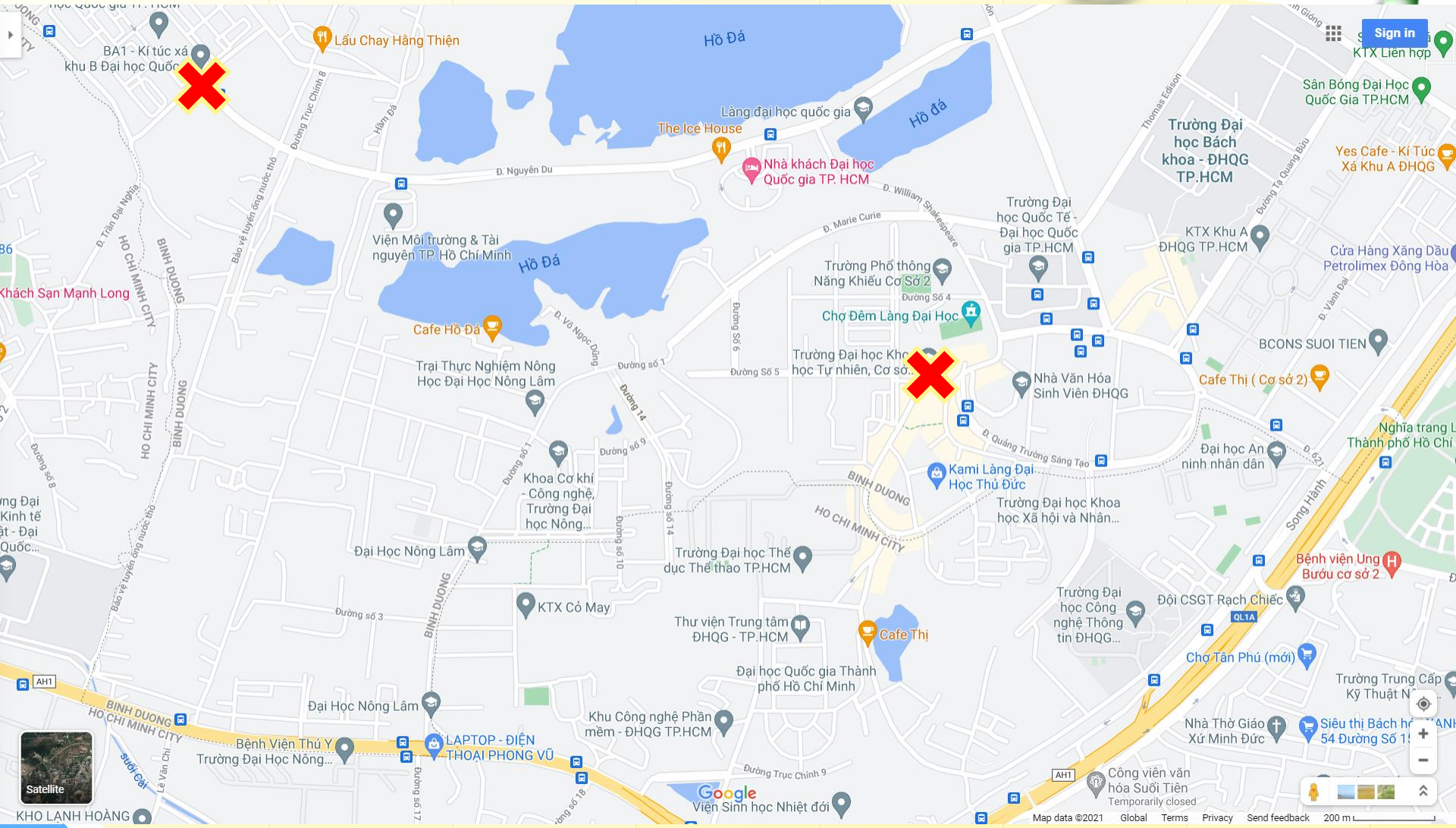
Nội dung

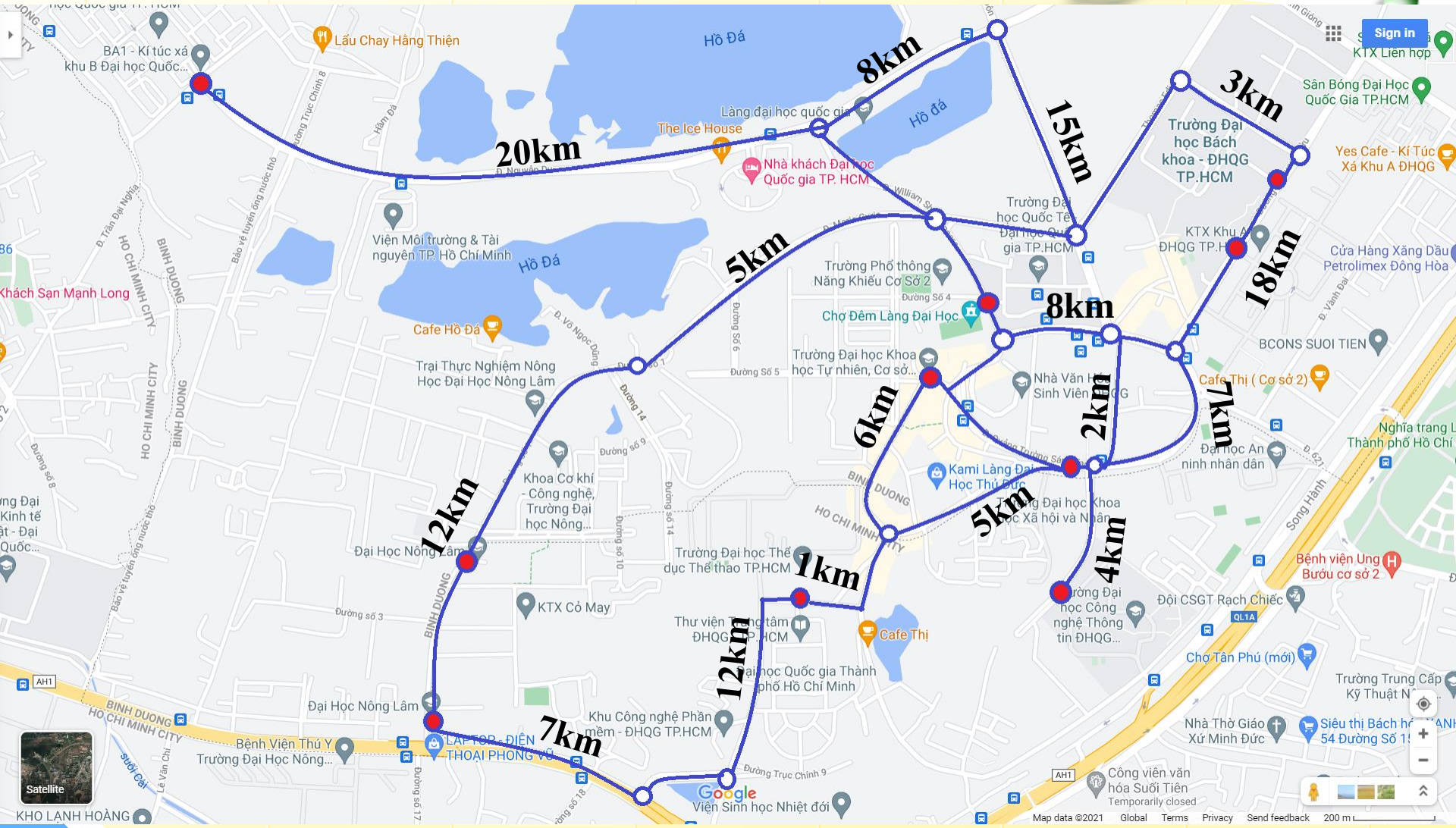
6.1 Các khái niệm trên đồ thị

6.2 Biểu diễn đồ thị trên máy tính

6.3 Duyệt đồ thị

6.4 Thuật toán tìm đường đi ngắn nhất


BA1 - Kí túc xá khu B Đại học Quốc
Lẩu Chay Hằng Thiện
Hồ Đá
Làng đại học quốc gia
The Ice House
Nhà khách Đại học Quốc gia TP. HCM
Đ. Nguyễn Du
Đường Trục Chính 8
Hầm Đá
Viện Môi trường & Tài nguyên TP. Hồ Chí Minh
Cafe Hồ Đá
Trại Thực Nghiệm Nông Học Đại Học Nông Lâm
Khoa Cơ khí - Công nghệ, Trường Đại học Nông...
Đại Học Nông Lâm
KTX Cỏ May
Đường số 3
Đại Học Nông Lâm
Bệnh Viện Thú Y Trường Đại Học Nông...
LAPTOP - ĐIỆN THOẠI PHONG VŨ
Suối Cái
KHO LẠNH HOÀNG
Khách Sạn Mạnh Long
Đ. Trần Đại Nghĩa
HO CHI MINH CITY
BINH DUONG
Bảo vệ tuyến ống nước thô
Đường số 8
Đường số 1
Đường số 9
Đường số 14
Đường số 10
Đường số 17
Đường số 18
Đường Trục Chính 9
Đ. Võ Ngọc Dũng
Đ. Marie Curie
Đ. William Shakespeare
Đường Số 4
Đường Số 5
Đường Số 6
Trường Phổ thông Năng Khiếu Cơ Sở 2
Trường Đại học Quốc Tế - Đại học Quốc gia TP.HCM
Chợ Đêm Làng Đại Học
Trường Đại học Khoa học Tự nhiên, Cơ sở...
Nhà Văn Hóa Sinh Viên ĐHQG
Đ. Quảng Trường Sáng Tạo
Kami Làng Đại Học Thủ Đức
Trường Đại học Khoa học Xã hội và Nhân...
Trường Đại học Thể dục Thể thao TP.HCM
Thư viện Trung tâm ĐHQG - TP.HCM
Cafe Thị
Đại học Quốc gia Thành phố Hồ Chí Minh
Khu Công nghệ Phần mềm - ĐHQG TP.HCM
Viện Sinh học Nhiệt đới
Trường Đại học Bách khoa - ĐHQG TP.HCM
Đ. Tạ Quang Bửu
Thomas Edison
KTX Khu A ĐHQG TP.HCM
Yes Cafe - Kí Túc Xá Khu A ĐHQG
Sân Bóng Đại Học Quốc Gia TP.HCM
KTX Liên hợp
Cửa Hàng Xăng Dầu Petrolimex Đông Hòa
Đ. Vành Đai
BCONS SUOI TIEN
Cafe Thị (Cơ sở 2)
Đại học An ninh nhân dân
Đ. 622
Nghĩa trang L Thành phố Hồ Chí
Song Hành
Bệnh viện Ung Bướu cơ sở 2
Đội CSGT Rạch Chiếc
QL1A
Trường Đại học Công nghệ Thông tin ĐHQG...
Chợ Tân Phú (mới)
Trường Trung Cấp Kỹ Thuật N...
Nhà Thờ Giáo Xứ Minh Đức
Siêu thị Bách h... 54 Đường Số 15
Công viên văn hóa Suối Tiên
Temporarily closed
AH1
Satellite
Sign in
Google
Map data ©2021
Global
Terms
Privacy
Send feedback
200 m


20km
8km
15km
3km
18km
8km
5km
6km
2km
7km
12km
1km
5km
4km
12km
7km
Hồ Đá
Hồ đá
Lẩu Chay Hằng Thiện
BA1 - Kí túc xá khu B Đại học Quốc...
Làng đại học quốc gia
The Ice House
Nhà khách Đại học Quốc gia TP. HCM
Viện Môi trường & Tài nguyên TP. Hồ Chí Minh
Trường Đại học Bách khoa - ĐHQG TP.HCM
Trường Đại học Quốc tế - Đại học Quốc gia TP.HCM
Trường Phổ thông Năng Khiếu Cơ Sở 2
Chợ Đêm Làng Đại Học
Cafe Hồ Đá
Trại Thực Nghiệm Nông Học Đại Học Nông Lâm
Trường Đại học Khoa học Tự nhiên, Cơ sở...
Khoa Cơ khí - Công nghệ, Trường Đại học Nông...
Đại Học Nông Lâm
Trường Đại học Thể dục Thể thao TP.HCM
KTX Cỏ May
Thư viện Trung tâm ĐHQG TP.HCM
Cafe Thị
Khu Công nghệ Phần mềm - ĐHQG TP.HCM
Đại học Quốc gia Thành phố Hồ Chí Minh
Bệnh Viện Thú Y Trường Đại Học Nông...
LAPTOP - ĐIỆN THOẠI PHONG VŨ
Viện Sinh học Nhiệt đới
KHO LẠNH HOÀNG
Sân Bóng Đại Học Quốc Gia TP.HCM
Yes Cafe - Kí Túc Xá Khu A ĐHQG
KTX Khu A ĐHQG TP.H...
Cửa Hàng Xăng Dầu Petrolimex Đông Hòa
BCONS SUOI TIEN
Cafe Thị (Cơ sở 2)
Nhà Văn H... Sinh Viên ...G
Kami Làng Đại Học Thủ Đức
Đại học An ninh nhân dân
Bệnh viện Ung Bướu cơ sở 2
Đội CSGT Rạch Chiếc
Chợ Tân Phú (mới)
Nhà Thờ Giáo Xứ Minh Đức
Công viên văn hóa Suối Tiên
Temporarily closed
Satellite
Map data ©2021
Global
Terms
Privacy
Send feedback
200 m
Sign in

Các bài toán

1. Tìm đường đi ngắn nhất từ một đỉnh đến một đỉnh khác.
2. Tìm đường đi ngắn nhất từ một đỉnh đến các đỉnh còn lại
3. Tìm đường đi ngắn nhất giữa các cặp đỉnh

Thuật toán Dijkstra

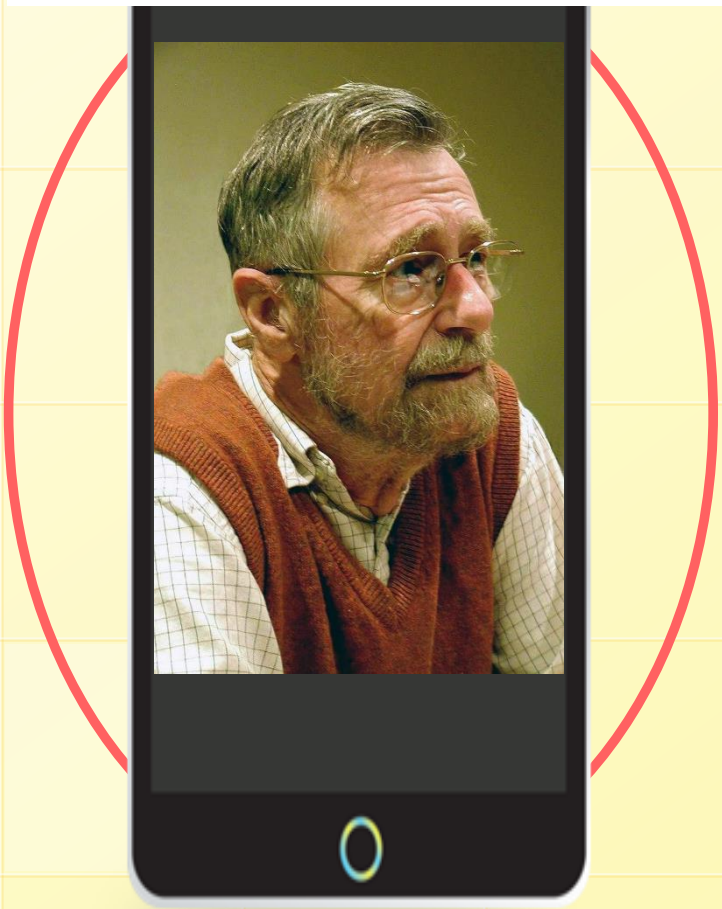
❖ Giới thiệu:

- Thuật toán tìm đường đi ngắn nhất từ một đỉnh xuất phát đến tất cả các đỉnh còn lại trong đồ thị có hướng hoặc vô hướng, có trọng số không âm
- Được công bố lần đầu vào năm 1959
- Được đặt tên theo tên của nhà toán học và nhà vật lý người Hà Lan Edsger W. Dijkstra

Edsger Wybe Dijkstra

(/ˈdaɪkstrə/ *DYKE-strə*)

(11/5/1930 – 6/8/2002)



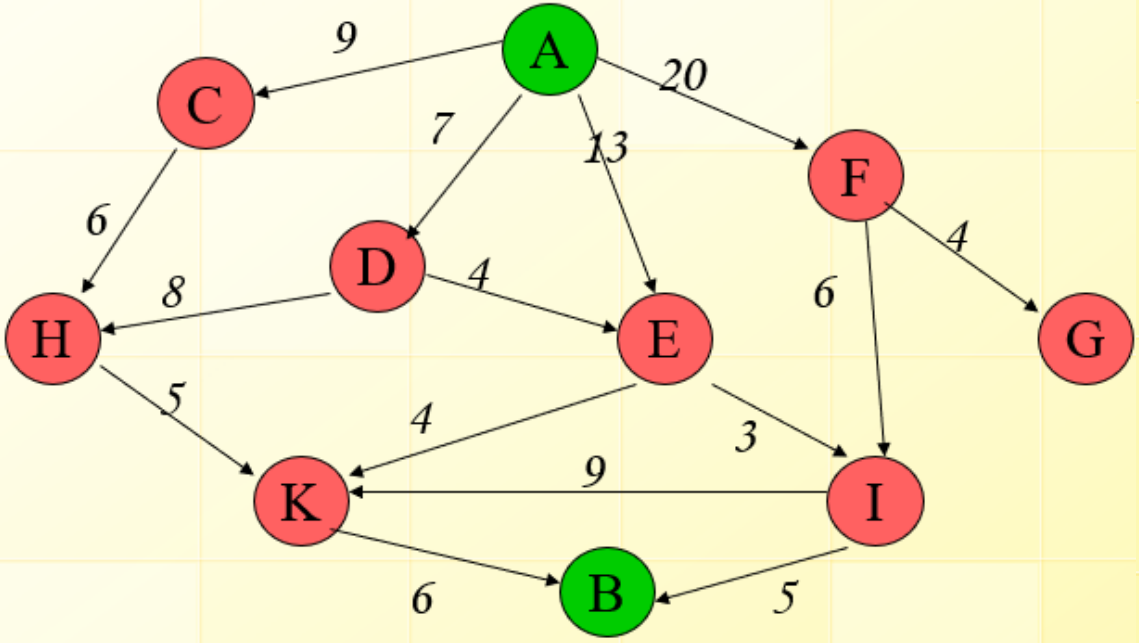
- Nhà toán học, nhà vật lý, nhà khoa học máy tính người Hà Lan
- Làm việc tại Trung tâm toán học, Viện nghiên cứu quốc gia về toán học và khoa học máy tính tại Amsterdam
- Giữ chức vị giáo sư tại Đại học Kỹ thuật Eindhoven, Hà Lan.
- Nghỉ hưu năm 2000

- Có các đóng góp mang tính chất nền tảng trong lĩnh vực ngôn ngữ lập trình
- Thuật toán Dijkstra, hệ điều hành THE và cấu trúc semaphore…
- *"Computer Science is no more about computers than astronomy is about telescopes"*

Thuật toán Dijkstra

❖ Bài toán 1:

- **Input:** Đồ thị có hướng G=(V,E,w), w(i,j) ≥ 0 ∀ (i,j)∈E, đỉnh nguồn s, đỉnh đích g
- **Output:** đường đi ngắn nhất từ đỉnh s đến đỉnh g và độ dài của đường đi



Thuật toán Dijkstra

❖**Ý tưởng**:

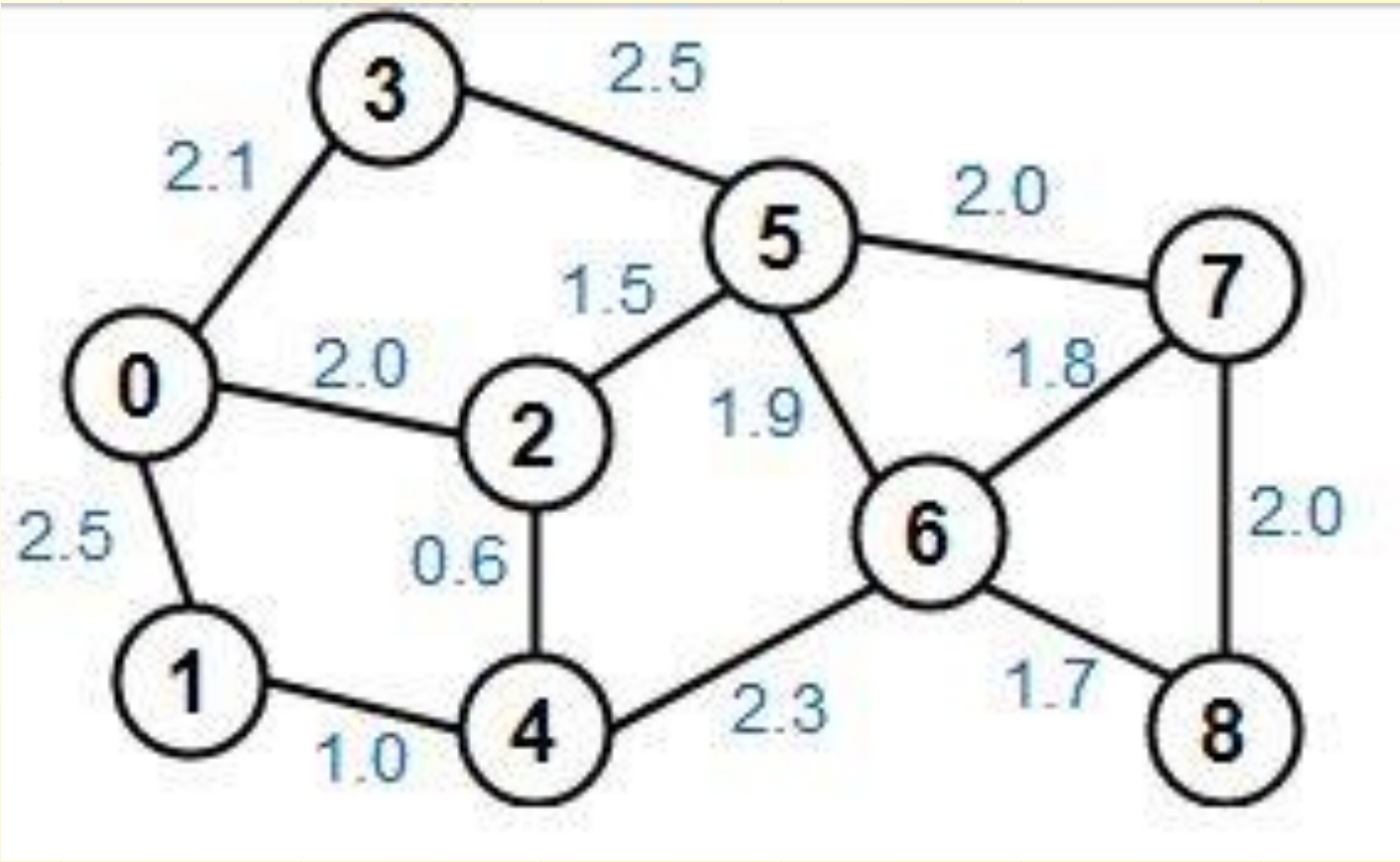
- **Bước 1:** Từ đỉnh gốc, khởi tạo khoảng cách tới chính nó là 0, khởi tạo khoảng cách nhỏ nhất ban đầu tới các đỉnh khác là +∞. Ta được danh sách các khoảng cách tới các đỉnh.
- **Bước 2:** chọn đỉnh a có khoảng cách nhỏ nhất trong danh sách này và ghi nhận. Các lần sau sẽ không xét đến đỉnh này nữa.

Thuật toán Dijkstra

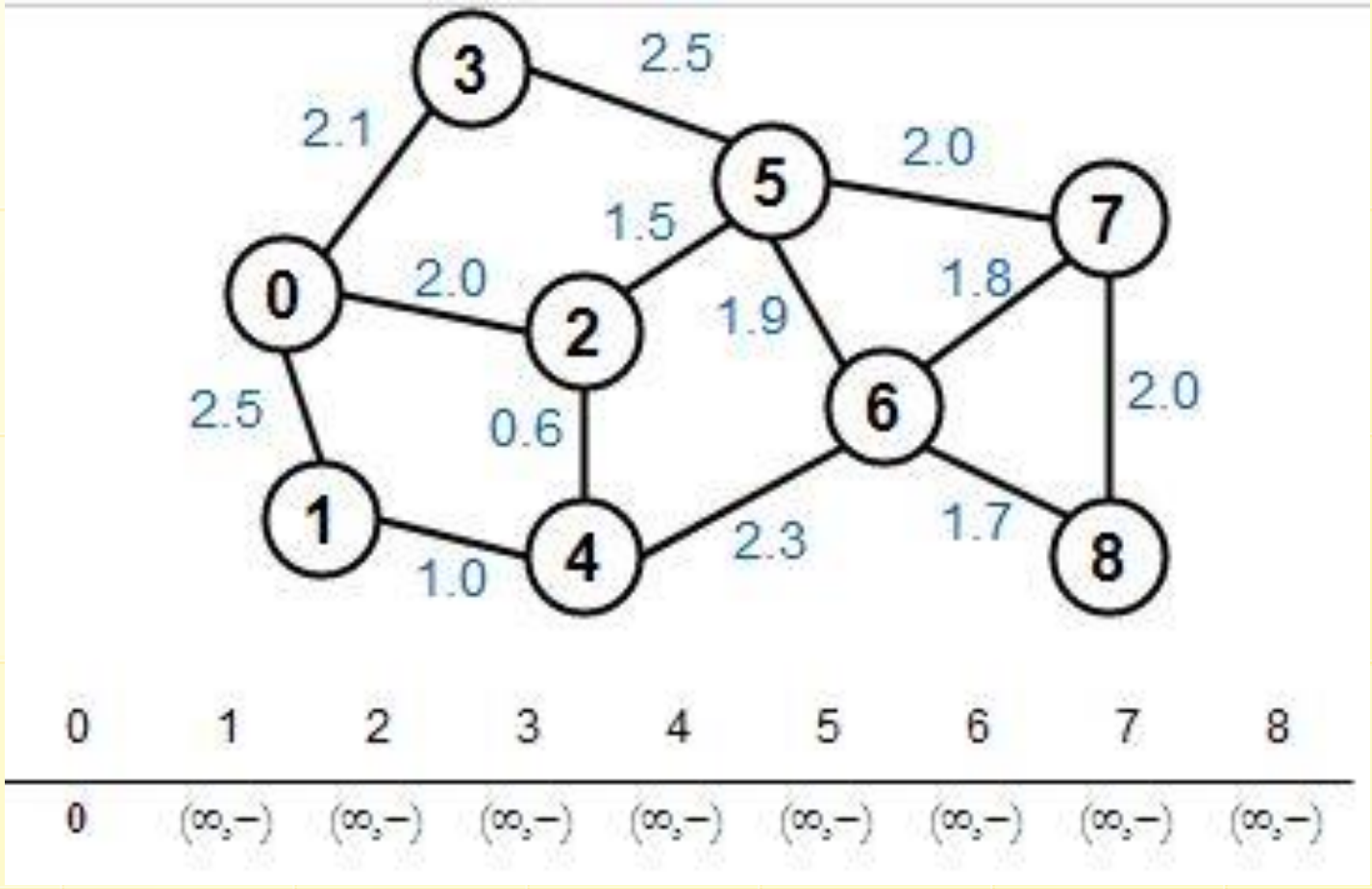
- **Bước 3:** Lần lượt xét các đỉnh kề b của đỉnh a. Nếu khoảng cách từ đỉnh gốc đến đỉnh b nhỏ hơn khoảng cách hiện tại đang được ghi nhận thì cập nhật giá trị và đỉnh kề a khoảng cách hiện tại của b.
- **Bước 4:** Sau khi xét tất cả đỉnh kề b của đỉnh a. Lúc này ta được danh sách khoảng cách tới các điểm đã được cập nhật. Quay lại ***Bước 2*** với danh sách này. Thuật toán kết thúc khi chọn được khoảng cách nhỏ nhất từ tất cả các điểm.

Ví dụ

Xét đồ thị vô hướng G. Thuật toán Dijkstra tìm khoảng cách từ đỉnh gốc 0 tới tất cả các đỉnh còn lại trong đồ thị G.

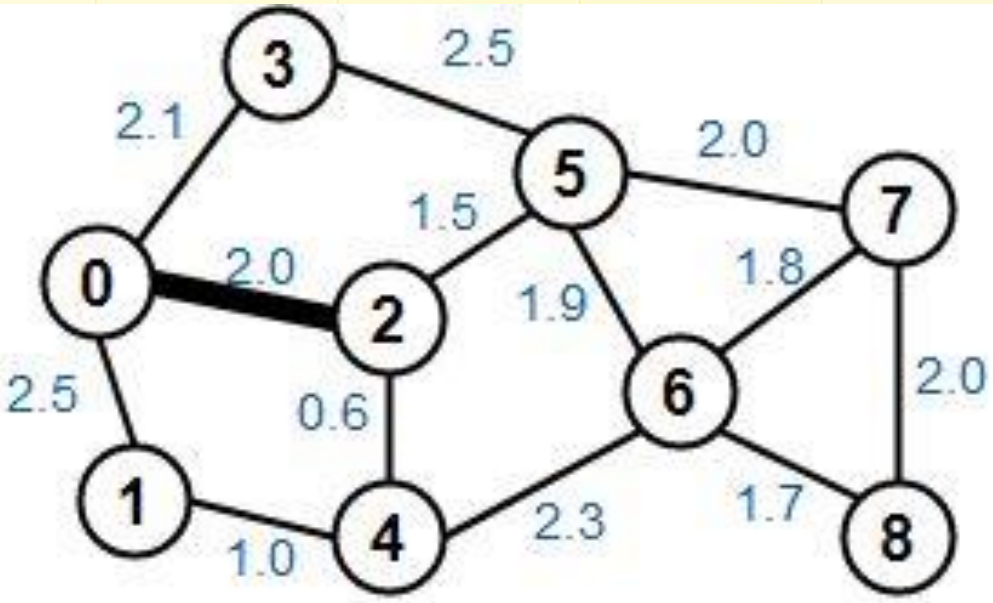


Đầu tiên, khởi tạo khoảng cách nhỏ nhất ban đầu tới các đỉnh khác là +∞ và khoảng cách tới đỉnh gốc là 0. Ta được danh sách các khoảng cách tới các đỉnh.



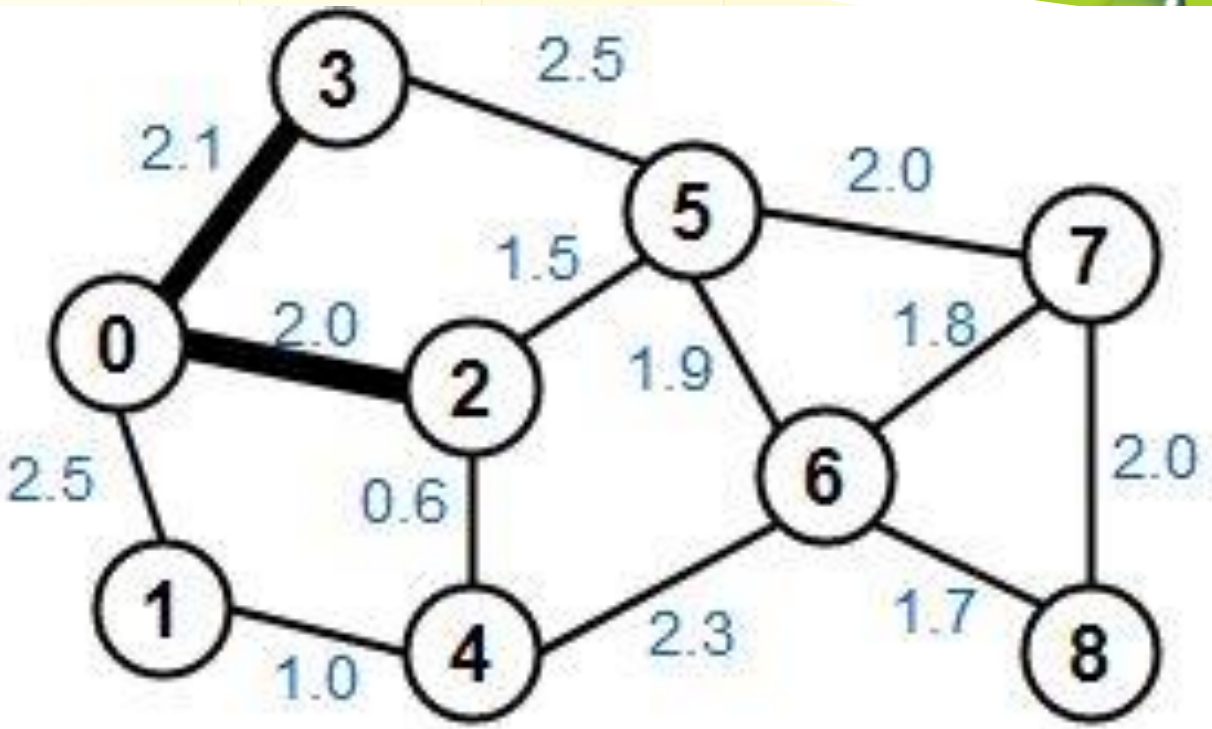
0	1	2	3	4	5	6	7	8
0	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)

Chọn đỉnh 0 có giá trị nhỏ nhất, xét các đỉnh kề của đỉnh 0: Xét đỉnh 1, khoảng cách từ gốc đến đỉnh 1 là 2.5 < ∞ nên ghi nhận giá trị mới là (2.5,0) (nghĩa là khoảng cách đến đỉnh gốc hiện tại ghi nhận là 2.5, đỉnh kề liền trước là đỉnh 0). Xét tương tự cho đỉnh 2 và 3, ta được dòng thứ 2 trong bảng.



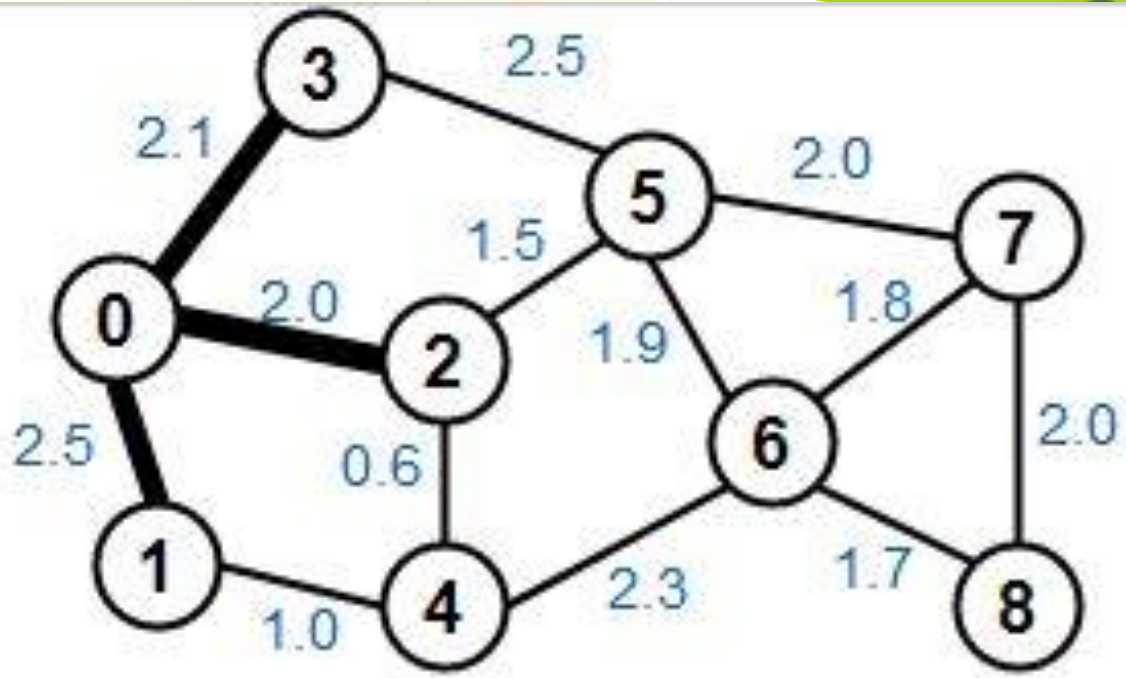
0	1	2	3	4	5	6	7	8
0	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)
–	(2.5, 0)	**(2.0, 0)**	(2.1, 0)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)

Sau khi xét tất cả các đỉnh ta chọn đỉnh 2 có khoảng cách nhỏ nhất và ghi nhận để xét tiếp. Tiếp tục xét đỉnh kề của 2 là đỉnh 4 và 5 với nguyên tắc nêu ở trên. Xét đỉnh 4, khoảng cách từ đỉnh gốc đến đỉnh 4 sẽ bằng khoảng cách từ đỉnh gốc tới đỉnh 2 cộng khoảng cách từ 2 đến 4. Nghĩa là 2.0+0.6=2.6 nên ta ghi nhận khoảng cách tại đỉnh 4 là (2.6,2). Xét tương tự cho đỉnh 5.



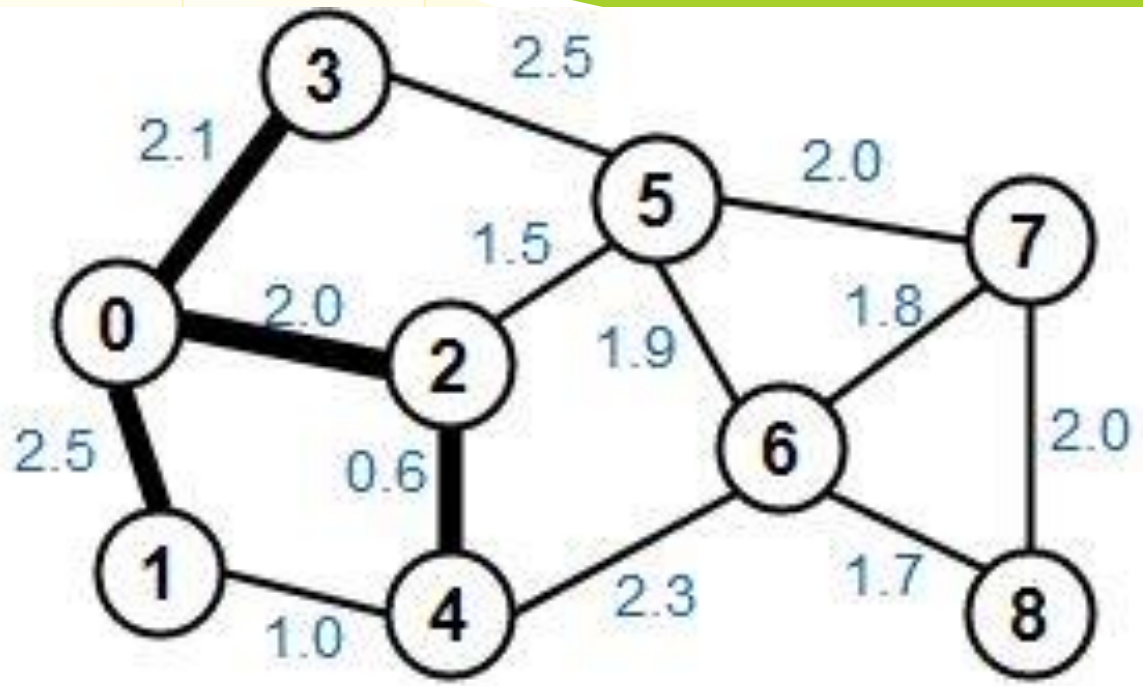
0	1	2	3	4	5	6	7	8
0	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)
–	(2.5, 0)	**(2.0, 0)**	(2.1, 0)	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)
–	(2.5, 0)	–	**(2.1, 0)**	(2.6, 2)	(3.5, 2)	(∞, –)	(∞, –)	(∞, –)

Lúc này ta chọn được đỉnh 3 có khoảng cách nhỏ nhất, xét đỉnh kề của đỉnh 3 là đỉnh 5. Khoảng cách từ gốc tới đỉnh 5=2.1+2.5=4.6 lớn hơn khoảng cách hiện tại được ghi nhận, vì vậy giá trị tại đỉnh 5 không đổi.



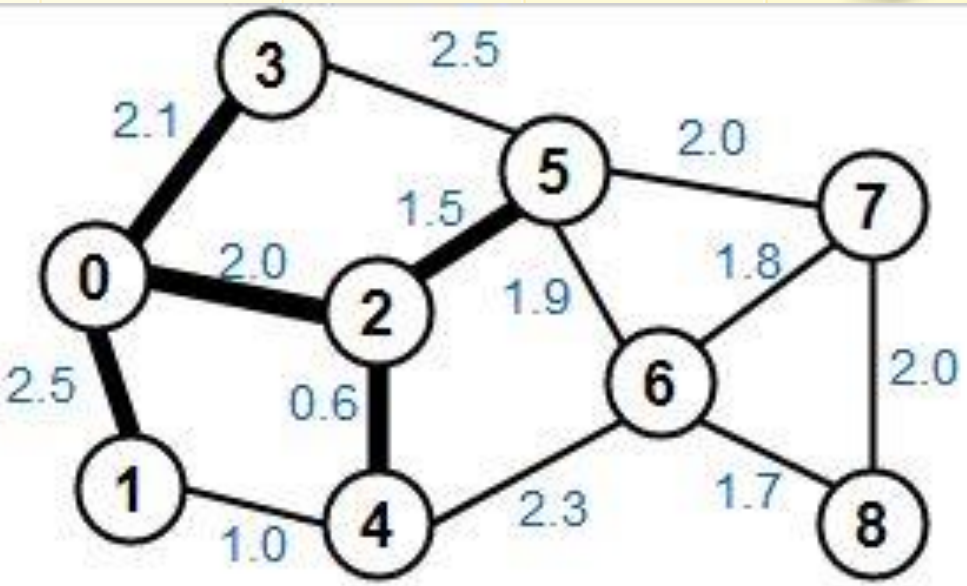
0	1	2	3	4	5	6	7	8
0	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)
–	(2.5, 0)	**(2.0, 0)**	(2.1, 0)	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)
–	(2.5, 0)	–	**(2.1, 0)**	(2.6, 2)	(3.5, 2)	(∞, –)	(∞, –)	(∞, –)
–	**(2.5, 0)**	–	–	(2.6, 2)	(3.5, 2)	(∞, –)	(∞, –)	(∞, –)

Đỉnh 1 là đỉnh được chọn tiếp theo, xét đỉnh kề của 1 là đỉnh 4. Khoảng cách từ đỉnh gốc không nhỏ hơn khoảng cách hiện tại nên ta không cập nhật gì ở đỉnh này.



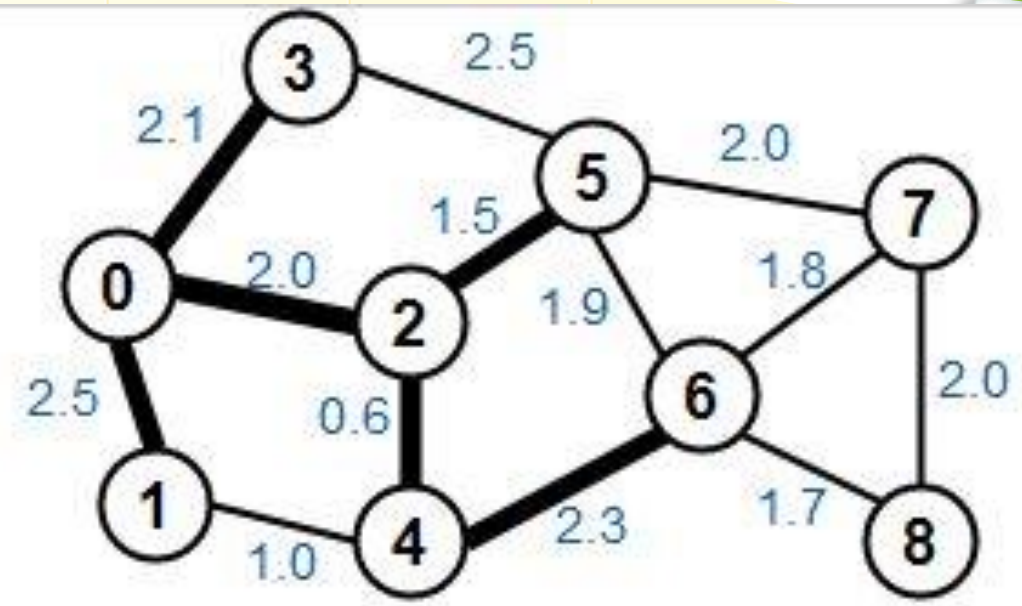
0	1	2	3	4	5	6	7	8
0	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)
–	(2.5, 0)	**(2.0, 0)**	(2.1, 0)	(∞, –)	(∞, –)	(∞, –)	(∞, –)	(∞, –)
–	(2.5, 0)	–	**(2.1, 0)**	(2.6, 2)	(3.5, 2)	(∞, –)	(∞, –)	(∞, –)
–	**(2.5, 0)**	–	–	(2.6, 2)	(3.5, 2)	(∞, –)	(∞, –)	(∞, –)
–	–	–	–	**(2.6, 2)**	(3.5, 2)	(∞, –)	(∞, –)	(∞, –)

Sau khi xét xong ta chọn được đỉnh 4 là đỉnh tiếp theo. Ta cập nhật giá trị mới cho đỉnh 6.



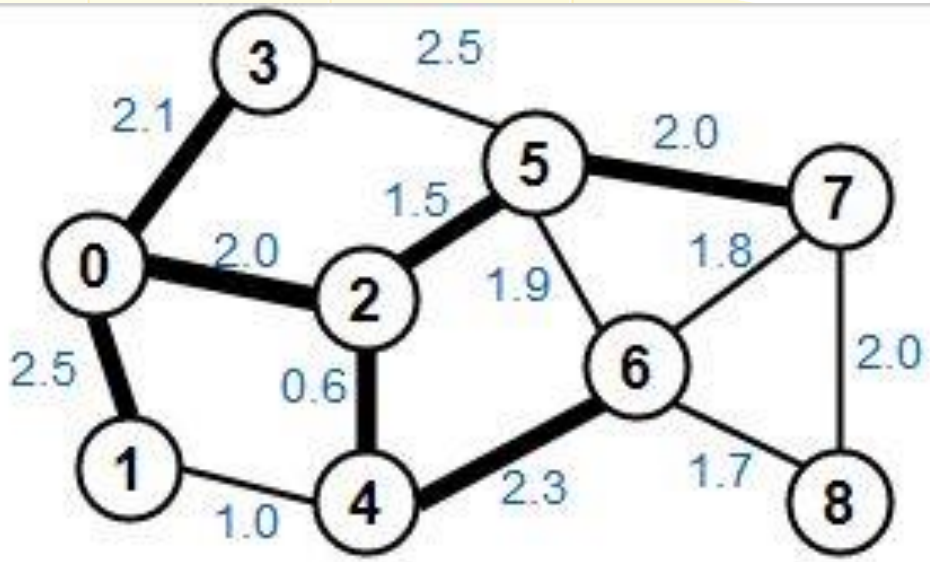
0	1	2	3	4	5	6	7	8
0	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)
–	(2.5, 0)	**(2.0, 0)**	(2.1, 0)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)
–	(2.5, 0)	–	**(2.1, 0)**	(2.6, 2)	(3.5, 2)	(∞,–)	(∞,–)	(∞,–)
–	**(2.5, 0)**	–	–	(2.6, 2)	(3.5, 2)	(∞,–)	(∞,–)	(∞,–)
–	–	–	–	**(2.6, 2)**	(3.5, 2)	(∞,–)	(∞,–)	(∞,–)
–	–	–	–	–	**(3.5, 2)**	(4.9, 4)	(∞,–)	(∞,–)

Chọn được đỉnh 5 là đỉnh nhỏ nhất, tiếp tục xét các đỉnh kề.



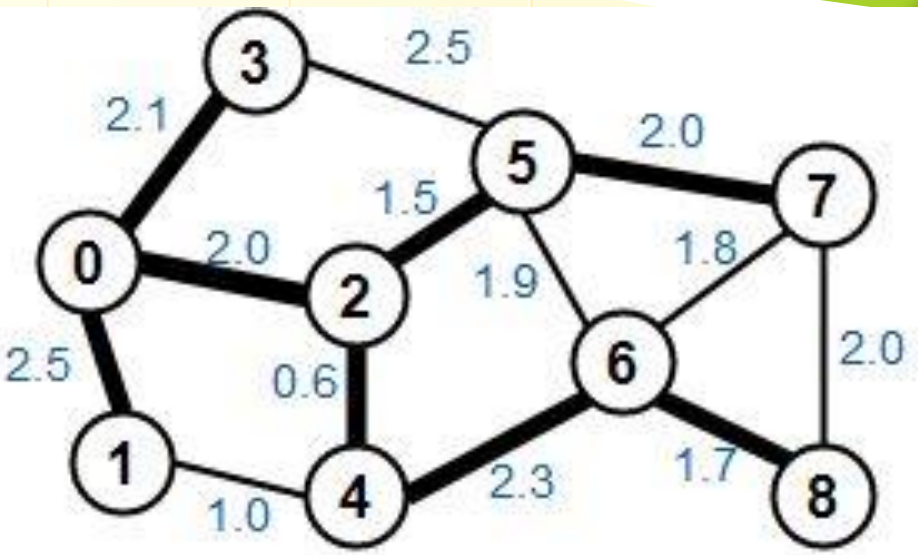
0	1	2	3	4	5	6	7	8
0	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)
–	(2.5, 0)	**(2.0, 0)**	(2.1, 0)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)
–	(2.5, 0)	–	**(2.1, 0)**	(2.6, 2)	(3.5, 2)	(∞,–)	(∞,–)	(∞,–)
–	**(2.5, 0)**	–	–	(2.6, 2)	(3.5, 2)	(∞,–)	(∞,–)	(∞,–)
–	–	–	–	**(2.6, 2)**	(3.5, 2)	(∞,–)	(∞,–)	(∞,–)
–	–	–	–	–	**(3.5, 2)**	(4.9, 4)	(∞,–)	(∞,–)
–	–	–	–	–	–	**(4.9, 4)**	(5.5, 5)	(∞,–)

Đỉnh 6 là đỉnh tiếp theo được chọn.



0	1	2	3	4	5	6	7	8
0	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)
–	(2.5, 0)	**(2.0, 0)**	(2.1, 0)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)
–	(2.5, 0)	–	**(2.1, 0)**	(2.6, 2)	(3.5, 2)	(∞,–)	(∞,–)	(∞,–)
–	**(2.5, 0)**	–	–	(2.6, 2)	(3.5, 2)	(∞,–)	(∞,–)	(∞,–)
–	–	–	–	**(2.6, 2)**	(3.5, 2)	(∞,–)	(∞,–)	(∞,–)
–	–	–	–	–	**(3.5, 2)**	(4.9, 4)	(∞,–)	(∞,–)
–	–	–	–	–	–	**(4.9, 4)**	(5.5, 5)	(∞,–)
–	–	–	–	–	–	–	**(5.5, 5)**	(6.6, 6)

Chọn đỉnh có khoảng cách nhỏ nhất là đỉnh 7.



0	1	2	3	4	5	6	7	8
0	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)
–	(2.5, 0)	**(2.0, 0)**	(2.1, 0)	(∞,–)	(∞,–)	(∞,–)	(∞,–)	(∞,–)
–	(2.5, 0)	–	**(2.1, 0)**	(2.6, 2)	(3.5, 2)	(∞,–)	(∞,–)	(∞,–)
–	**(2.5, 0)**	–	–	(2.6, 2)	(3.5, 2)	(∞,–)	(∞,–)	(∞,–)
–	–	–	–	**(2.6, 2)**	(3.5, 2)	(∞,–)	(∞,–)	(∞,–)
–	–	–	–	–	**(3.5, 2)**	(4.9, 4)	(∞,–)	(∞,–)
–	–	–	–	–	–	**(4.9, 4)**	(5.5, 5)	(∞,–)
–	–	–	–	–	–	–	**(5.5, 5)**	(6.6, 6)
–	–	–	–	–	–	–	–	**(6.6, 6)**

Thuật toán Dijkstra

❖ Ý tưởng chính:

- Hàm d(u) dùng để lưu trữ độ dài đường đi (khoảng cách) ngắn nhất từ đỉnh nguồn s đến đỉnh u

$$d(u) = \min\{d(v) + w(v,u), v \in X(u)\}$$

$X(u)$ là tập tất cả các đỉnh có cạnh đi tới đỉnh u

- Đặt khoảng cách từ đỉnh nguồn s đến chính nó là 0 và đến tất cả các đỉnh khác là vô cùng
- Tiến hành lặp cho đến khi tất cả các đỉnh đã được xác định khoảng cách ngắn nhất từ s hoặc không còn đỉnh nào có thể đạt tới từ s
- Mỗi lần lặp, chọn đỉnh p chưa đi qua có giá trị d(p) nhỏ nhất, cập nhật khoảng cách của các đỉnh kề thông qua đỉnh được chọn p

Ký hiệu

Gọi Open : tập các đỉnh có thể xét ở bước kế tiếp, các đỉnh có thể được xem xét lại, đỉnh chờ duyệt

Close : tập các đỉnh đã xét/đã duyệt, không xem xét lại

s : đỉnh xuất phát

g : đỉnh đích

p : đỉnh đang xét, đỉnh hiện hành

$\Gamma(p)$: tập các đỉnh kề của p

$q \in \Gamma(p)$: một đỉnh kề của p

Dijkstra

❖ Giải thuật tham khảo:

Mỗi đỉnh p tương ứng với 1 số d(p): khoảng cách đi từ đỉnh ban đầu tới p

Bước 1: Khởi tạo

Open := {s};

Close :={};

d(s) :=0;

Bước 2: While (Open ≠ {})

2.1 Chọn p thuộc Open có d(p) nhỏ nhất

2.2 Nếu p là trạng thái kết thúc thì xuất đường đi, thoát

2.3 Nếu p đã duyệt rồi thì bỏ qua, trở lại đầu vòng lặp

2.4 Chuyển p qua Close, và **mở các đỉnh kế tiếp q sau p (kề với p)**

d(q) = d(p) + cost(p,q) ;

Thêm q vào Open

Bước 3: Không có kết quả.

Tìm kiếm Dijkstra

Bước 2: While (**Open ≠{}**)

2.4 Chuyển p qua Close, và mở các đỉnh kế tiếp q sau p

2.3.1 Nếu q đã có trong Open

nếu d(q)> d(p)+cost(p,q)

d(q) = d(p) + cost(p,q);

parent(q)=p ;

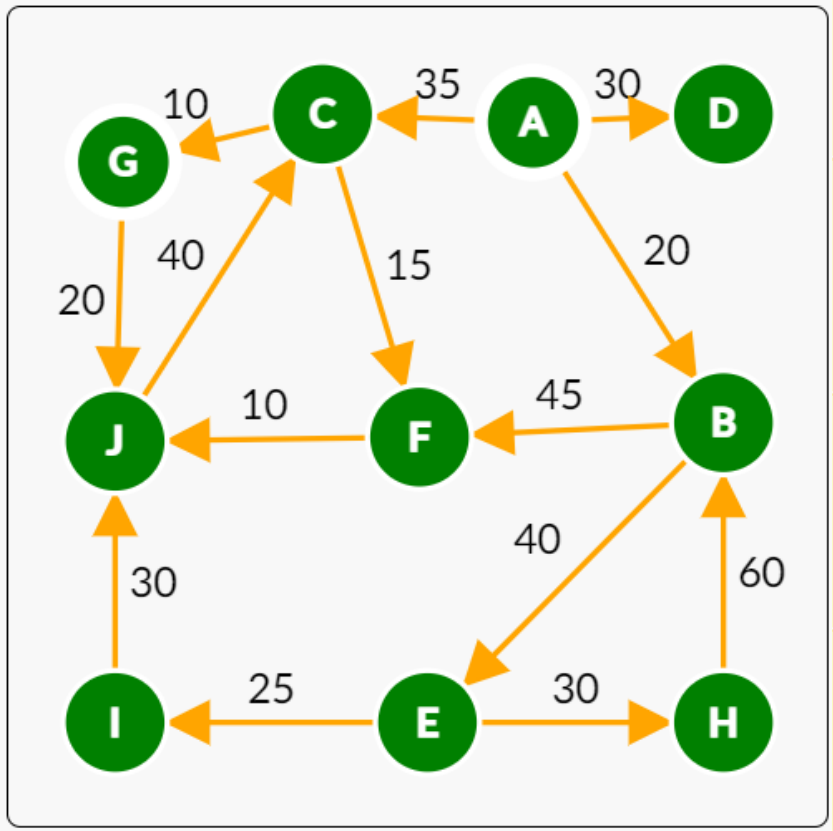
2.3.2 Nếu q chưa có trong Open và Close

d(q) = d(p) +cost(p,q);

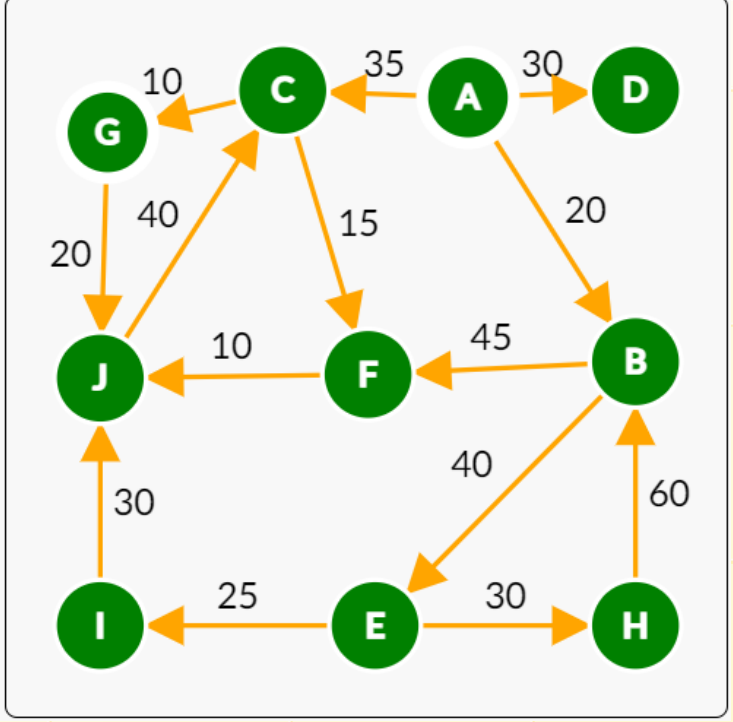
parent(q)=p;

Thêm q vào Open

Ví dụ: Tìm đường đi ngắn nhất từ A đến J



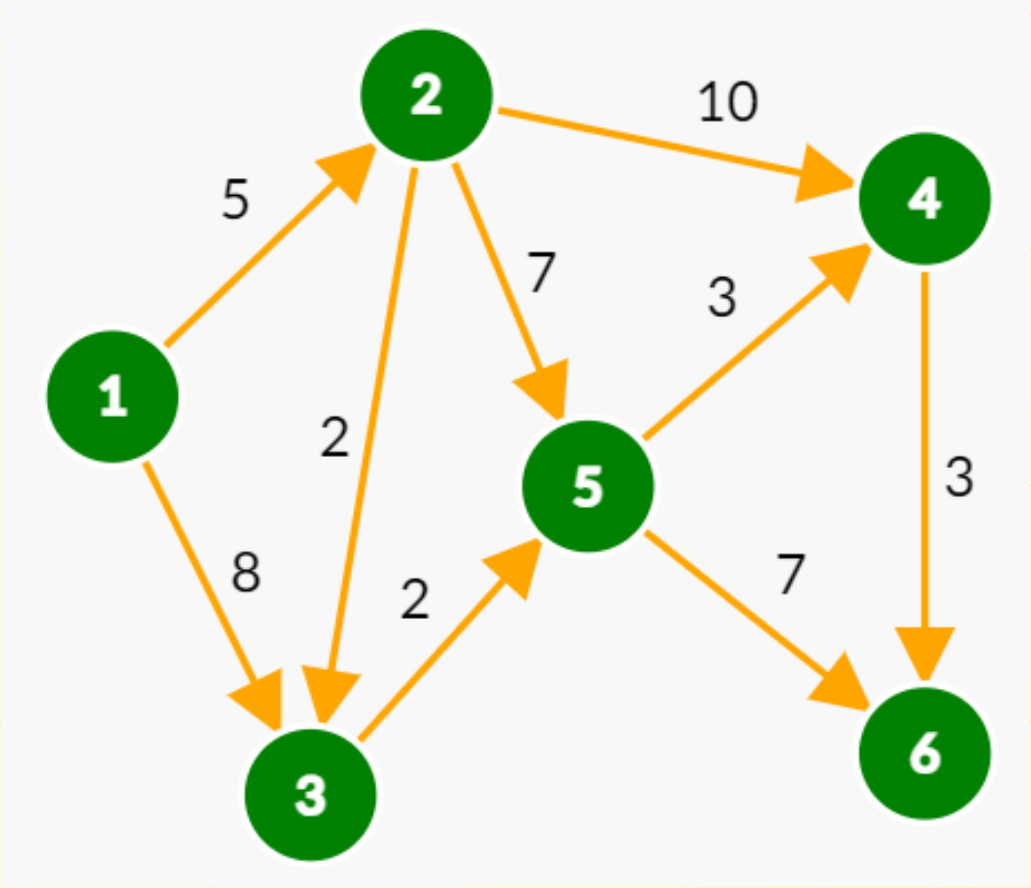

```
10 14
A B C D E F G H I J
A B 20
A C 35
A D 30
B E 40
B F 45
C F 15
C G 10
E H 30
E I 25
F J 10
G J 20
H B 60
I J 30
J C 40
```



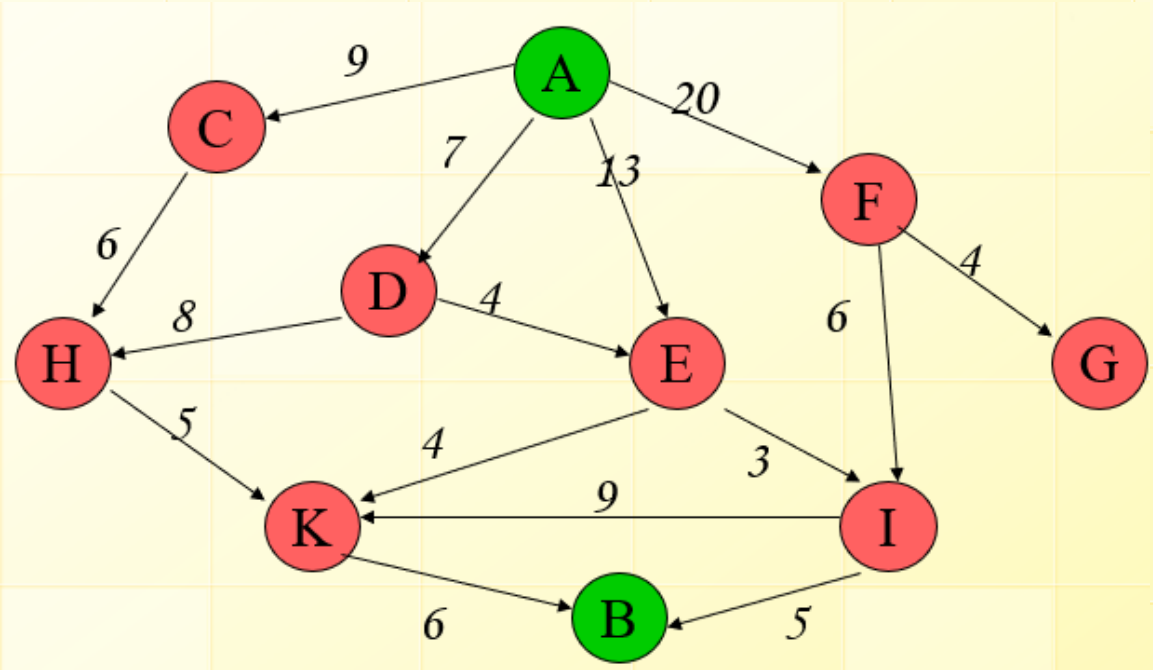
Ví dụ: Tìm đường đi ngắn nhất từ A đến J dùng Dijkstra

p	Γ(p)	Open	Close
		{(A,0)}	{}
A	{B,C,D}	{(B,20), (C,35), (D,30)}	{A}
B	{E,F}	{(C,35), (D,30), (E,60), (F,65)}	{A,B}
D	{}	{(C,35), (E,60), (F,65)}	{A,B,D}
C	{F,G}	{(E,60), ~~(F,65)~~, (F,50), (G,45)}	{A,B,D,C}
G	{J}	{(E,60), (F,50), (J,65)}	{A,B,D,C,G}
F	{J}	{(E,60), ~~(J,65)~~, (J,60)}	{A,B,D,C,G,F}
J			

Bài tập tại lớp:
Tìm đường đi ngắn nhất từ 1 đến 6



Bài tập tại lớp:
Tìm đường đi ngắn nhất từ A đến B



Tìm kiếm Dijkstra

❖ Một số gợi ý về cài đặt:

1. Tổ chức cấu trúc dữ liệu

- vector<vector<int> > matrix; // ma trận trọng số của đồ thị
- vector<string> v_list; // lưu danh sách các tên đỉnh (chuỗi)
- map<string, int> v_index; // ánh xạ từ tên đỉnh sang index để có thể truy xuất thông tin trong ma trận kề
- priority_queue<….>open; // lưu các đỉnh chờ duyệt
- vector<bool> close(v, 0); // lưu thông tin đỉnh nào đã duyệt qua rồi
- map<string, string> parent; // lưu thông tin cha con, parent[u]=v nghĩa là cha của u và v
- vector<int> d(v,INF); // lưu khoảng cách (ngắn nhất) từ s đến các đỉnh khác

Priority Queue

Khởi tạo min heap từ priority queue

```
priority_queue< pair<int,int>, vector<pair<int,int>>,greater<pair<int,int>>> open;
```

Các hàm trên Priority Queue:

- size(): Trả về kích thước của hàng đợi
- empty(): kiểm tra có rỗng hay không
- top(): Lấy phần tử có độ ưu tiên cao nhất
- pop(): Xóa phần tử có độ ưu tiên cao nhất
- push(): Thêm 1 phần tử vào

Priority queue cũng giống như queue nhưng được thiết kế đặc biệt để phần tử ở đầu luôn luôn là phần tử có độ ưu tiên lớn nhất so với các phần tử khác

Tìm kiếm Dijkstra

❖ Một số gợi ý về cài đặt:

Bước 1: open.push({0,s}); d[s] = 0;

Bước 2: while (!open.empty())

```
        pair<int, int> top = open.top();  open.pop();
        int p=top.second, d=top.first;
        if (p == g) {found = true; break; }
        if(close[p]==1) continue;
    close[p] = 1;
        for (int i = 0; i < v; i++)
            if (matrix[p][i]>0 && close[i]== 0)
                if (d[i]>d[p]+ matrix[p][i] )
                    d[i]=d[p]+ matrix[p][i] ; open.push({d[i],i});  parent[i] = p;
```

Bước 3: Xử lý xuất đường đi hoặc kết luận không tìm thấy